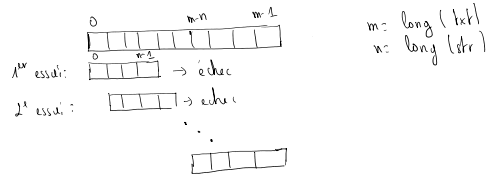


Contexte : * Dans la recherche classique d'une str dans un texte (txt),

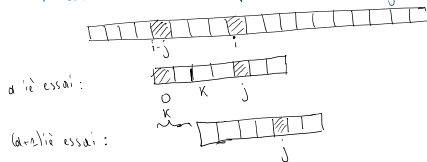


au pire des cas on se retrouve à faire $n \times (m-n+1)$ comparaisons

* L'objectif ainsi à atteindre est de linéariser cette complexité pour l'avoir en $O(m+n)$: Soit l'algorithme KMP

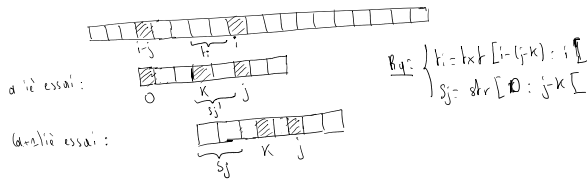
Idees : * On veut exploiter les comparaisons déjà faites qui ont conduit à des presque-égales pour ne plus avoir à seulement décaler d'une position après un échec

$\rightarrow$ cas d'échec à une position (i, j) ($j \geq 1$) (ce qui se répète tant qu'on ne trouve pas str)



* On va tenter de shifter str de K caractères qui ne sera plus nécessairement 1, mais à quelle contrainte ?

N'oublions pas qu'on veut linéariser la complexité et une idée intéressante serait de faire en sorte que la prochaine comparaison au niveau de txt, demeure à l'indice d'échec i ; ainsi soit on avance sur txt soit on reste sur place ; on ne fait jamais de retour en arrière. C'est déjà quelque chose. voyons ce que ça donne



Pour que la comparaison continue en position i, il faut que les portions t_i et s_j coïncident et ainsi on n'aura qu'à continuer la comparaison avec $txt[i]$ et $str[j]$

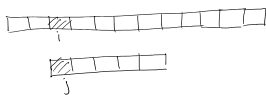
Or n'oublions pas qu'au 1^{er} essai $txt[i:j] = str[0:j]$ coïncidaient et donc en particulier s_j' et k .

Ainsi $s_j' = k - s_j$; s_j' est ainsi un ps (prefixe-suffixe) de $str[0:j]$; la contrainte qu'on cherche pointe déjà son neg.

Mieux en core un bon shift de str caractérisé par K est tel que $k - s_j$ soit le plus long possible, à fin qu'il ne manque presque plus de caractères de str à comparer, augmentant ainsi nos chances d'avoir le motif str cherché dans txt.

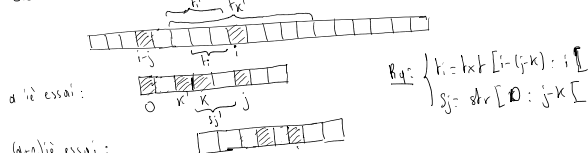
Il vient naturellement qu'il faut prendre en considération le plus gros ps de $str[0:j]$. On veut ainsi minimiser $j-k$ et donc minimiser par ricochet K. Or $K \geq 1 \Rightarrow$ ce ps est strict

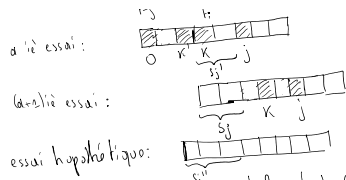
$\rightarrow$ cas d'échec en (i, j) ($j \geq 0$)



Dans ce cas il faut juste décaler str d'une position vers la droite ; ça revient tout simplement à incrémenter i dans le processus.

Justification : * Montrons qu'on ne peut avoir roté une présence de str dans txt qu'on aurait obtenu en shiftant str de seulement $K' < K$ car a d'ores





$$Hq: \begin{cases} k = \text{txt}[i-j+1] : i \\ s_j = \text{str}[0] : j-k \end{cases}$$

Si tel était le cas, on aurait $\text{str} = \text{txt}$ et donc en particulier $s_j = i$. En raisonnant comme on l'a fait plus haut, on constate qu'on vient de déceler un ps de $\text{str}[0:j]$ plus long que k ; ce qui est absurde au vu de la condition finale sur k .

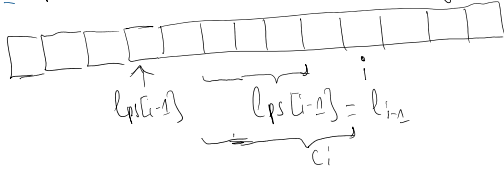
* Le processus de construction de l'idée du plus long

ps (lps: longest prefix suffix) assure que l'avancée dans txt se fait dans un seul sens. Mais vu qu'on peut stagner à une position i , est-ce qu'on aura vraiment une complexité en $O(n+m)$? On verra dans la suite.

principe: * il faut d'abord s'entraîner en mesure d'évaluer les $\text{lps}[i] = \text{lps de str}[0:i]$, $\forall i \in \mathbb{N}, i \leq n$ et sans excéder $O(n)$. Ici par lps on entendra la longueur de ce plus long ps.

Hq: Je suis allé jusqu'à n car je me suis dit que ce serait intéressant de essayer de voir si on ne peut pas une fois avoir toutes les occurrences; en effet ce cas correspond à la situation où $j = n$ et donc on a trouvé une occurrence de str , mais on aimerait poursuivre la recherche comme si de rien n'était. La raison est en amont s'applique et la situation est équivalente à chercher $\text{lps}[n]$.

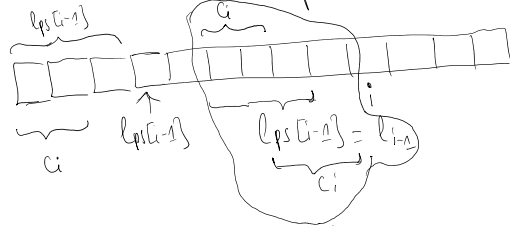
- $\text{lps}[1] = 0$
- Pour $i \geq 2$ ($i \leq n$) / qu'on a déjà évalué les $\text{lps}[j]_{j=1}^{i-1}$ on a:



$$Hq: \text{lps}[k] = l_k$$

Le meilleur candidat est $C_i = \text{str}[i-l_{i-1}-1 : i]$.

S'il ne marche pas alors $l_i \leq l_{i-1}$.



Comme le montre la partie encadrée, la recherche de l_i s'effectue maintenant dans $\text{str}[l_{i-1} : i]$.

Cette fois-ci le meilleur candidat est le lps de l_{i-1} car le lps de i s'il n'est pas inférieur en taille à celui là, en lui retirant un caractère, il devient le lps de l_{i-1} .

Ainsi, formellement,

Pour i allant de 2 à n faire

$l \leftarrow \text{lps}[i-1]$

Tant que $l > 0$ et $\text{str}[l-1] \neq \text{str}[l]$ faire

$l \leftarrow \text{lps}[l]$; // On décroît strict vu que $\text{lps}[l] \leq l-1$

Fin Tant que

Fin Tant que

Si $\text{str}[i-1] \neq \text{str}[l]$ Faire

$\text{lps}[l] = \text{lps}[l] + 1$

Sinon

$\text{lps}[l] = 0$

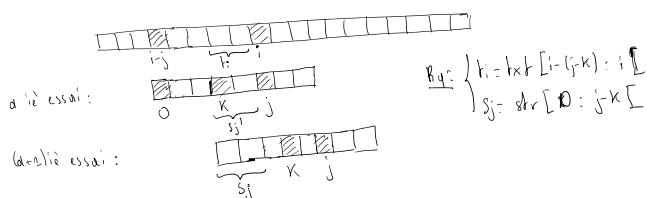
Fin Si

Fin bouc

→ Complexité: (En nbre de comparaisons)

On part du constat que $\text{lps}[k] \geq 0 \quad \forall k \in \mathbb{I}_1; n$
Tant qu'on reste dans l'une des boucles, l ne reste pas stable
Le nbre d'itérations est ainsi entièrement déterminé par l
Au cours des phases itératives, l commence à 0 et est incrémenté
On constate qu'au cours d'une itération (while ou for)
l est incrémenté au max 1 fois et ce de 1 (indirectement via $\text{lps}[i-1]$)
Or vu que l commence à 0 et que pour s'exécute $n-1$ fois
et ainsi l est incrémenté au plus $n-1$ fois dans tout le programme
De plus car $l \geq 0$, l sera décrémenté au plus $n-1$ fois également
ce qui limite le nbre d'itérations de la boucle à $n-1$ au plus
On se retrouve ainsi avec au plus $n-1 + n-1 = 2n-2 = O(n)$ comparaisons
D'où $O(n)$

* Ayant lps il faut maintenant rechercher tous les motifs comme
convenu plus haut



De la figure ci-dessous après un échec en position i ou une
occurrence entièrement trouvée, il faudra reprendre la comparaison
de $\text{txt}[l]$ avec $\text{str}[d]$ où $d = \text{lps}[j]$

On obtient ainsi le code ci-dessous

$i \leftarrow 0; j \leftarrow 0;$

Tant que $i \leq m-n$ Faire

Tant que $\text{str}[j] = \text{txt}[i]$ Faire

Tant que $i \leq m-n$ Faire

Tant que $str[j] = txt[i]$ Faire

$i \leftarrow i+1$;
 $j \leftarrow j+1$;

FinTantQue

Si $j = 0$ Alors

$i \leftarrow i+1$;

Sinon

Si $j = n$ Alors

occ. appends $[i-j]$;

FinSi

$j \leftarrow lps[j]$;

FinSi

FinTantQue

→ Complexité: (en terme de comparaisons)

On a autant de conditions que de variation de j et de i dans " $j=0$ ".

- Ds le 1^{er} cas j commence à 0 et j est incrémenté seulement si i l'est aussi. or i commence à 0 et $i \leq m-n$ ainsi j est incrémenté au plus $m-n$ fois ; et est donc également décrémenté au plus $m-n$ fois dans tout le processus, soit au plus $2m-2n$ variations de j .

- Ds le 2^e cas, i est incrémenté ; mais ce cas peut être compté ds le 1^{er} car avec le 1^{er} on a exactement $m-n$ incréments de i en tout (les incréments de j étant comptés) et au plus $m-n$ décréments de j comme expliqué plus haut vu que $j \geq 0$

Soit une complexité en tout de $C_2 = m-n + m-n = 2m-2n$.

En conclusion la complexité, $C = C_1 + C_2 = 2m-2n + 2n-2 = 2m-2 = O(m)$